NED University of Engineering and Technology
(Department of Computer Science and Information Systems)

CT-363 – Design and Analysis of Algorithms
Complex Engineering Problem (CEP)

Student Name: **Faculty Signature:**

Student ID: **Date:**

CLOs	PLOs	Bloom Taxonomy
CLO-2	PLO-02: Problem Analysis	C3: Applying
CLO-3	PLO-03: Design/Development of Solution	C4: Analyzing

SNo.	Complex Engineering Solving Attributes	Excellent (75–100%)	Average (50–75%)	Poor (<50%)	Marks
CLO-2 (10%)	**WP1 – Depth of Engineering Knowledge** Resolved with forefront in-depth engineering knowledge	States both real-world sub-problems clearly and links each to the correct algorithmic paradigm with sound theoretical justification.	Defines the problems but the mapping to paradigms or the theoretical grounding is incomplete.	Cannot define the problems or lacks the knowledge to connect them to any algorithmic technique.	
CLO-3 (20%)	**WP2 – Range of Conflicting Requirements** Involves wide-ranging / conflicting technical, economic and safety issues	Correctly reconciles conflicting constraints (cost vs. coverage vs. time-decay vs. capacity) and justifies the trade-off chosen.	Identifies the conflicting constraints but resolves them only partially or inconsistently.	Ignores conflicting constraints or treats the problem as if no trade-offs exist.	
CLO-3 (30%)	**WP3 – Depth of Analysis Required** No obvious solution; requires abstract thinking and originality to formulate suitable models	Formulates a correct model for each paradigm (greedy-choice / optimal-substructure proof) and derives the correct time & space complexity with justification.	Formulates a mostly correct model with minor errors in the optimality argument or complexity derivation.	Unable to identify a workable model; complexity analysis is missing or wrong.	
CLO-2 (40%)	**WP5 – Extent of Applicable Code** Beyond codes of practice; correct, tested, and efficient implementation	Correctly and efficiently implements both algorithms (plus the bonus approximation module if attempted), passes all provided and edge-case test data, with clean pseudocode and code.	Implements the underlying logic but inefficiently, or fails some edge-case tests.	Completely unable to implement the required methodology correctly.	
				Total Marks:	**20**

Background

Algorithm design paradigms - brute force, divide-and-conquer, dynamic programming, greedy strategies, and heuristic / approximation methods - trade off optimality, running time, and implementation complexity differently depending on the structure of the underlying problem. Real engineering systems rarely need only one paradigm: large systems are usually decomposed into sub-problems that are each best solved with a different technique, and the sub-solutions are then combined.

In this assignment, you will design, analyze, and implement a single real-life decision-support system that genuinely requires two different paradigms working together. You must use at least two of the following paradigm families across the scenario: Divide-and-Conquer, Dynamic Programming, Greedy Algorithms, and Approximation/Heuristic Algorithms. (If you would instead like to attempt only one paradigm, it is only acceptable if applied to Part D - the NP-hard bonus case - at full depth; see the note at the end.)

Scenario: Flood-Relief Coordination System

A district struck by seasonal flooding has activated an emergency operations centre (EOC). The EOC must (A) determine which submerged roads can still be used to reach a set of affected villages at minimum total reconnection cost, and (B) decide how to pack a limited fleet of relief trucks with food, medicine and shelter kits so that the total humanitarian impact is maximized before flood waters rise further. Both decisions must be made from the same underlying village and road data, and both feed the same after-action report.

You will be given (or must generate, if no dataset is supplied by your instructor) the following:

- A set of n villages ($2 \le n \le 200$), each with GPS coordinates, a population count, and a severity score (1–10).
- A set of m candidate roads connecting village pairs, each with a repair/traversal cost and a boolean flag "currently submerged".
- A fleet of k relief trucks, each with a distinct weight capacity.
- A supply catalogue of items, each with a weight and a "lives-impacted" value; villages have differing per-item requirements.

Part A - Road-Network Reconnection (Greedy Paradigm)

Apply a Minimum Spanning Tree greedy algorithm (Kruskal's or Prim's - you must justify your choice) to determine the minimum-cost set of roads that reconnects the EOC to every reachable village.

Requirements:

1. Exclude every road flagged "currently submerged" from the candidate edge set before running your algorithm.
2. If the resulting graph is disconnected (some villages cannot be reached at all), your program must detect this, report the isolated villages by name, and compute a Minimum Spanning Forest instead of failing.
3. Two or more roads may have identical cost. Implement an explicit, documented tie-breaking rule (e.g., prefer the road serving the higher combined severity score) and justify why your rule is reasonable for a disaster-response setting.
4. Prove, in your report, that your chosen greedy algorithm satisfies the greedy-choice property and optimal-substructure property for this problem.
5. Derive the exact time complexity in terms of n (villages) and m (roads) for your chosen implementation (adjacency list + priority queue / disjoint-set with union-by-rank and path compression), and state the space complexity.
6. Extension case: after the initial MST is computed, three additional roads become submerged (simulating rising water). Your solution must recompute the affected part of the network without recomputing the entire MST from scratch - describe and implement an incremental update strategy, and compare its cost to a full recomputation.

Part B - Relief Supply Allocation (Dynamic Programming Paradigm)

Model the packing of relief supplies into the truck fleet as a Bounded/Multiple-Knapsack problem and solve it with dynamic programming to maximize total "lives-impacted" value under each truck's distinct weight capacity.

Requirements:

1. This is not a single 0/1 knapsack: you have k trucks with k different capacities, and each supply item may be loaded onto at most one truck in total. Design and justify a DP formulation (state definition, recurrence, base case) that correctly handles multiple knapsacks - do not simply run single-knapsack DP k times without accounting for shared item usage.
2. Every item has a fixed weight and a fixed lives-impacted value given directly in the input data (see the sample dataset). Your DP must treat these values as constants
3. Handle the edge case where total demand exceeds total truck capacity for every village (no feasible allocation gives every village 100% of its request) - your solution must still return the value-maximizing partial allocation, not fail or return zero.
4. Handle the edge case of items whose weight exceeds every truck's individual capacity (an item that can never be carried by any single truck) - such items must be automatically and correctly excluded from consideration without breaking the recurrence.
5. Reconstruct and print the actual item-to-truck assignment (not just the optimal value) by backtracking through your DP table.
6. Derive the exact time and space complexity of your multi-knapsack DP in terms of number of items, number of trucks, and total capacity, and explain why a brute-force enumeration of assignments is infeasible even for modest n by giving the brute-force complexity for comparison.

Part C - Integrated Analysis

Combine Part A and Part B into a single pipeline and answer the following in your report:

- A worked example on a dataset of at least 10 villages and 20 roads, showing the MST chosen, the final knapsack assignment, and the total lives-impacted value achieved.
- A sensitivity test: rerun the pipeline after removing your two highest-severity villages from the network, and discuss how both the greedy and DP outputs change.

Deliverables

- Problem formulation and assumptions (half a page).
- Pseudocode for every algorithm used, clearly labelled by part.
- Time and space complexity derivation for every algorithm used
- Working source code (any language agreed with your instructor) with the test dataset and at least 3 edge-case tests (disconnected graph, over-demand, an item too heavy for any single truck) demonstrated with output.
- A minimum 6-page written report covering Parts A–C. It also consists of (Title page, Abstract, Introduction, Literature Review, Problem formulation, Methodology, block diagram, Complexity Analysis, Results, Conclusion and Limitation, References).

NOTE: Any real-life application will be implemented using divide-and-conquer algorithms, approximation algorithms, Greedy Algorithms, and Dynamic Programming that are mentioned in the course Plan, but a case study needs to be developed first for any new project.